

WEEK 03

The Cloud Outage & Injection Case Study — Monitored RAG Service

Cloud Deployment, Retrieval & Production Security

REAL-WORLD INCIDENT

Incident: Global AI Cloud Infrastructure Outages & OWASP LLM01 Vulnerabilities (2026)

Source Reference: [OWASP GenAI Security Project — LLM01 Prompt Injection Standard](#)

Summary: Global AI cloud platform outages demonstrated the risk of relying solely on unmonitored external APIs. Simultaneously, indirect prompt injection inside ingested PDFs continues to hijack RAG vector retrievers.

Engineering Problem: Absence of input sanitization guardrails, lack of multi-provider fallback logic, and zero real-time telemetry into retrieval latency and token costs.

WHAT ENTERPRISES EXPECT

- Input sanitization wrappers to filter prompt injection attempts.
- Hybrid search (vector embeddings + BM25 keyword matching) for retrieval accuracy.
- Real-time observability dashboards tracking token usage, latency, and retrieval evaluation metrics.

DELIVERABLE

Deliverable	Technology Stack
Cloud-served RAG API with tracing, evals, and security controls.	Weaviate · Bedrock/GCP · Arize Phoenix

IMPLEMENTATION PLAN

1. Local Observability Setup: Install **Arize Phoenix** (`pip install arize-phoenix`) and launch the dashboard locally using `import phoenix as px; px.launch_app()`.
2. Vector Indexing & Hybrid Search: Provision a free **Weaviate Cloud Sandbox** cluster (or local Weaviate Docker container). Ingest PDF documents using local HuggingFace embeddings (**all-MiniLM-L6-v2**) and configure hybrid search (dense vectors + sparse BM25).
3. Security Guardrails: Write an input-sanitization wrapper that scans incoming user queries for known prompt injection patterns (e.g., *"Ignore previous instructions"*).
4. Tracing & Verification: Wrap the RAG pipeline with **Phoenix** tracing spans and verify real-time execution traces, token costs, and evaluation scores at **http://localhost:6006**.